



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular

Campo de Estudio

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO- EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): MINAYA GARCIA CRISTOPHER JEFFERSON GABRIEL ARIEL VELASCO MOLINA	
Asignatura: ESTRUCTURA DE DATOS	Calificación:
Fecha del informe: 15-09-2026	
Nº Práctica: 4	
Título de la Práctica: Implementación y representación de árboles y grafos	
Introducción	La presente práctica experimental aplica los conocimientos de la Unidad IV mediante la implementación de un grafo dirigido y ponderado para resolver un problema tecnológico: encontrar vuelos baratos entre ciudades. La red de transporte se modela con vértices que representan aeropuertos y aristas que representan vuelos; cada arista almacena un precio y una duración. Esta representación permite organizar conexiones, consultar rutas y calcular el itinerario de menor costo. En teoría de grafos, una red puede representarse mediante un conjunto de vértices y un conjunto de aristas que conectan pares de vértices. En el caso desarrollado, el grafo es dirigido porque una ruta de origen a destino no implica necesariamente la existencia del trayecto inverso, y es ponderado porque cada arista tiene un costo numérico. Esta modelación es útil para navegación, logística y redes de transporte. Alamoudi y Al-Hashimi (2024) describen el problema de caminos mínimos desde una fuente como una tarea fundamental con aplicaciones en navegación, distribución de bienes y redes de comunicación [1]. La solución utiliza el algoritmo de Dijkstra, apropiado cuando los pesos son no negativos. El algoritmo mantiene una frontera de candidatos mediante una cola de prioridad y actualiza progresivamente la mejor distancia conocida. En Python, el módulo heapq implementa una cola de prioridad basada en un min-heap, cuyo elemento menor se mantiene en la raíz [2]. La literatura reciente también evidencia que Dijkstra se emplea para determinar rutas óptimas y que su tiempo depende del tamaño y densidad del grafo [3].

	La práctica se justifica porque un buscador de vuelos requiere comparar alternativas y seleccionar una ruta económicamente conveniente. Frente a un listado plano de vuelos, el grafo expresa directamente las conexiones y permite calcular caminos con escalas. El objetivo consiste en desarrollar una solución funcional que cargue una base ficticia desde un archivo de texto, consulte el grafo, reporte sus vértices y aristas, encuentre la ruta más barata, mida el tiempo de ejecución y analice las ventajas y desventajas de la estructura utilizada.																		
Desarrollo o procedimiento	Tema seleccionado y escenario Se seleccionó exclusivamente el tema “Encuentro de vuelos baratos a partir de una base de datos”. La actividad se desarrolló en un entorno de escritorio virtual/local, utilizando Python 3.11, terminal y archivos de texto. El escenario corresponde a un laboratorio de programación: se diseñó la estructura, se creó una base ficticia de rutas, se implementó el algoritmo, se ejecutaron pruebas y se generaron evidencias. No se emplearon datos personales ni se realizó una conexión con aerolíneas reales. Representación del grafo <table><tr><th>Elemento</th><th>Implementación</th><th>Función</th></tr><tr><td>Vértice</td><td>Código de aeropuerto: UIO, GYE, CUE, BOG, MDE, LIM, SCL</td><td>Representa una ciudad o aeropuerto.</td></tr><tr><td>Arista dirigida</td><td>Objeto Vuelo con origen y destino</td><td>Representa una conexión disponible en una dirección.</td></tr><tr><td>Peso principal</td><td>Precio en dólares</td><td>Permite minimizar el costo total del itinerario.</td></tr><tr><td>Atributo adicional</td><td>Duración en minutos</td><td>Se reporta como información complementaria.</td></tr><tr><td>Lista de adyacencia</td><td>diccionario: ciudad -> lista de vuelos</td><td>Organiza las salidas de cada vértice y evita recorrer rutas no conectadas.</td></tr></table> El archivo vuelos.txt contiene 16 aristas dirigidas y 7 vértices. Cada línea presenta origen, destino, precio y duración separados por el carácter . La clase GrafoVuelos carga los registros, construye la lista de adyacencia y expone operaciones de reportería y consulta. Procedimiento aplicado Primero se definió el modelo de datos y se registró una base ficticia de vuelos. Después se implementó la carga desde archivo y la consulta de ciudades. Para encontrar la ruta más barata se inicializan las distancias con infinito, se asigna cero al origen y se inserta el origen en una cola de prioridad. En cada iteración se extrae la ciudad con menor costo, se revisan sus vuelos salientes y se relajan las distancias cuando se encuentra un costo menor. Finalmente se reconstruye la ruta usando el diccionario de predecesores. La reportería permite visualizar el número de vértices, el número de aristas	Elemento	Implementación	Función	Vértice	Código de aeropuerto: UIO, GYE, CUE, BOG, MDE, LIM, SCL	Representa una ciudad o aeropuerto.	Arista dirigida	Objeto Vuelo con origen y destino	Representa una conexión disponible en una dirección.	Peso principal	Precio en dólares	Permite minimizar el costo total del itinerario.	Atributo adicional	Duración en minutos	Se reporta como información complementaria.	Lista de adyacencia	diccionario: ciudad -> lista de vuelos	Organiza las salidas de cada vértice y evita recorrer rutas no conectadas.
	Elemento	Implementación	Función																
	Vértice	Código de aeropuerto: UIO, GYE, CUE, BOG, MDE, LIM, SCL	Representa una ciudad o aeropuerto.																
	Arista dirigida	Objeto Vuelo con origen y destino	Representa una conexión disponible en una dirección.																
	Peso principal	Precio en dólares	Permite minimizar el costo total del itinerario.																
Atributo adicional	Duración en minutos	Se reporta como información complementaria.																	
Lista de adyacencia	diccionario: ciudad -> lista de vuelos	Organiza las salidas de cada vértice y evita recorrer rutas no conectadas.																	

	y la lista de adyacencia completa. La consulta específica solicita origen y destino, mientras que la consulta global ordena las rutas más baratas desde una ciudad. El benchmark repite búsquedas de Dijkstra para grafos de 7, 25, 50, 100 y 200 vértices y conserva los resultados en un archivo CSV. <i>Agente de IA utilizado</i> Se utilizó un agente de inteligencia artificial generativa como apoyo para proponer la estructura inicial del código, revisar la implementación del grafo, sugerir casos de prueba, organizar la documentación y mejorar la redacción técnica. La participación humana comprendió la selección del problema, adaptación de la base de datos, revisión de las salidas, corrección de una expectativa de prueba, ejecución y validación. Se estima que aproximadamente el 70 % del código recibió asistencia del agente y el 30 % correspondió a revisión, adaptación, ejecución y validación humana. Este porcentaje debe modificarse si la participación real del estudiante fue diferente.																
Resultados	<i>Resultados funcionales</i> La ejecución de demostración produjo un grafo con 7 ciudades y 16 rutas dirigidas. Para la consulta UIO–SCL, el sistema obtuvo la ruta UIO → CUE → SCL, con un precio total de 350 dólares y una duración acumulada de 315 minutos. La respuesta muestra que el algoritmo puede elegir una ruta con escala cuando el costo acumulado es menor que otras alternativas. <table border="1"> <thead> <tr> <th>Indicador</th><th>Resultado</th></tr> </thead> <tbody> <tr> <td>Vértices</td><td>7 ciudades/aeropuertos</td></tr> <tr> <td>Aristas</td><td>16 rutas dirigidas</td></tr> <tr> <td>Consulta principal</td><td>UIO → SCL</td></tr> <tr> <td>Ruta más barata</td><td>UIO → CUE → SCL</td></tr> <tr> <td>Costo mínimo</td><td>\$350.00</td></tr> <tr> <td>Duración reportada</td><td>315 minutos</td></tr> <tr> <td>Pruebas automatizadas</td><td>4 pruebas; resultado OK</td></tr> </tbody> </table> <i>Reportería y consultas</i> El reporte general presenta la lista de adyacencia, por ejemplo, UIO tiene conexiones hacia GYE por 55 dólares, CUE por 70 dólares y BOG por 180 dólares. La consulta de rutas desde UIO ordenó los destinos por costo: GYE (\$55), CUE (\$70), BOG (\$180), LIM (\$245), MDE (\$245) y SCL (\$350). Esta salida permite visualizar los elementos que conforman el grafo y verificar que la solución cumple la función de reportería solicitada. <i>Análisis del tiempo de ejecución</i> Con listas de adyacencia y una cola de prioridad basada en heap, la complejidad teórica de Dijkstra es $O((V + E) \log V)$, donde V representa vértices y E aristas. La ejecución local de 1000 búsquedas UIO–SCL tardó aproximadamente 0.0065 segundos. En el benchmark de grafos sintéticos controlados, el tiempo total de las repeticiones fue 0.00405 s para 7	Indicador	Resultado	Vértices	7 ciudades/aeropuertos	Aristas	16 rutas dirigidas	Consulta principal	UIO → SCL	Ruta más barata	UIO → CUE → SCL	Costo mínimo	\$350.00	Duración reportada	315 minutos	Pruebas automatizadas	4 pruebas; resultado OK
Indicador	Resultado																
Vértices	7 ciudades/aeropuertos																
Aristas	16 rutas dirigidas																
Consulta principal	UIO → SCL																
Ruta más barata	UIO → CUE → SCL																
Costo mínimo	\$350.00																
Duración reportada	315 minutos																
Pruebas automatizadas	4 pruebas; resultado OK																

vértices, 0.00256 s para 25, 0.00861 s para 50, 0.00521 s para 100 y 0.01157 s para 200. La variación entre puntos pequeños se debe a que se usaron diferentes cantidades de repeticiones para mantener el experimento manejable; el comportamiento general aumenta al crecer la estructura.

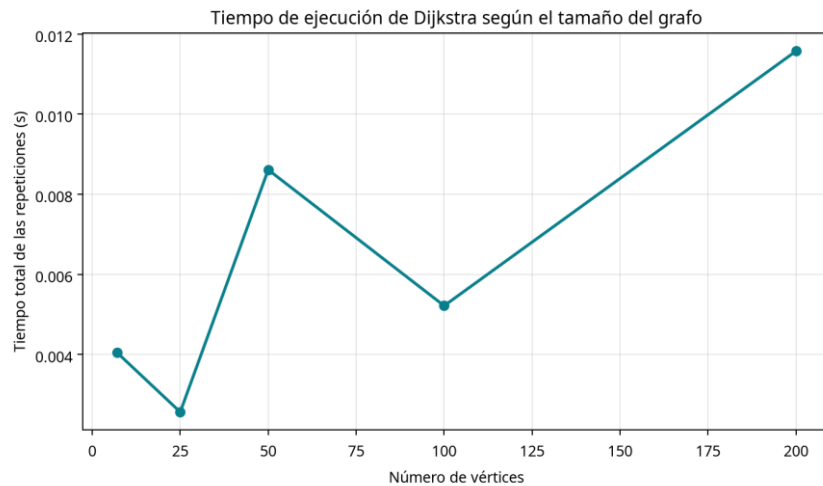


Figura 1. Tiempo de ejecución de Dijkstra según el tamaño del grafo.

Aspecto	Ventaja	Desventaja
Representación	La lista de adyacencia almacena eficientemente redes dispersas y hace explícitas las salidas de cada ciudad.	Una consulta requiere conocer la estructura y la ruta puede cambiar si se actualizan los vuelos.
Dijkstra	Encuentra el menor costo cuando todos los pesos son no negativos y ofrece una solución comprensible.	No es apropiado si existen pesos negativos y debe repetirse para cada origen si se necesitan muchas consultas.
Cola de prioridad	Permite extraer rápidamente el candidato con menor costo.	La implementación mantiene entradas obsoletas cuando una distancia mejora; estas se descartan al extraerlas.
Base ficticia	Facilita reproducibilidad y pruebas sin depender de servicios externos.	No representa horarios, impuestos, disponibilidad, equipaje ni cambios reales de precio.

Conclusiones

La práctica cumplió el objetivo de implementar un grafo dirigido y ponderado para resolver un problema tecnológico de búsqueda de vuelos baratos. La estructura permitió representar aeropuertos como vértices y conexiones como aristas con pesos de precio, conservando además la duración como atributo de consulta.

El algoritmo de Dijkstra encontró correctamente la ruta UIO → CUE → SCL con costo de 350 dólares y duración de 315 minutos. Las funciones de reportería permitieron visualizar los 7 vértices, las 16 aristas y las rutas más baratas desde un origen. Las cuatro pruebas automatizadas terminaron correctamente, lo que respalda el funcionamiento de la carga, la búsqueda, el ordenamiento y la validación de destinos inexistentes.

El análisis mostró que Dijkstra con lista de adyacencia y heap resulta adecuado para un prototipo con pesos no negativos, aunque su costo crece con el tamaño y la densidad del grafo. Como mejoras futuras se recomienda incorporar datos reales o una API, considerar horarios y

	escalas, agregar criterios multiobjetivo para equilibrar precio y duración, y utilizar A* o algoritmos especializados cuando exista una gran cantidad de aeropuertos y consultas.
Bibliografía	Alamoudi, O., & Al-Hashimi, M. (2024). On the energy behaviors of the Bellman–Ford and Dijkstra algorithms: A detailed empirical study. <i>Journal of Sensor and Actuator Networks</i>, 13(5), 67. https://doi.org/10.3390/jsan13050067 Python Software Foundation. (2026). heapq — Heap queue algorithm. Python Documentation. https://docs.python.org/3/library/heapq.html Wnorowski, J., & Łebkowski, A. (2024). The concept of determining a ship’s route based on the capability plot and Dijkstra’s algorithm—Finding the ship’s route between anchorages. <i>Applied Sciences</i>, 14(23), 11205. https://doi.org/10.3390/app142311205
Anexos	